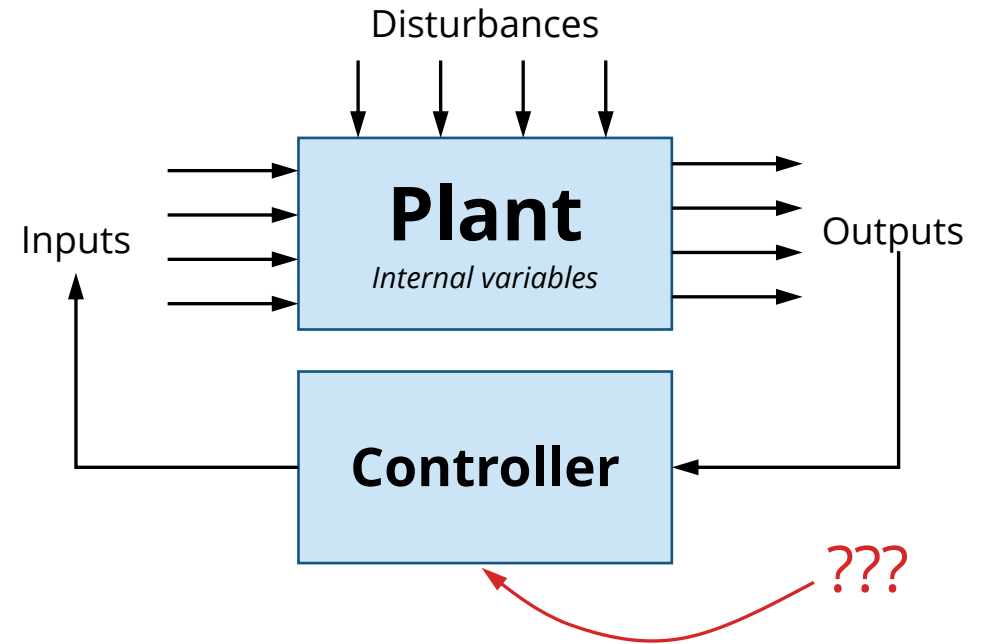




AA 513 Multivariable Control

Lecture 08 // Week 8

Instructor: Prof. Karen Leung

Announcements

- Research paper
 - Follow the PACES method. 1 (intro) +5 (PACES) slides. ~1 minute per slide.
 - Keep it high level. No need to get into the nitty-gritty details.
- Tutorial topic
 - Better to choose a simpler topic and focus on explaining/visualizing the concepts
 - Scope out the objectives of the tutorial. 1-2 learning objectives.
- Rubrics out (see EdDiscussion, Shiva will update on Canvas soon)
- Tutorial instructions updated. See website
 - Tutorial due Wednesday Dec 10 **Midnight**
 - Peer review due Monday Dec 15 **Noon (NO EXTENSIONS)**

Last time

- HJB equation – continuous time, continuous state and controls
- Linear quadratic regulator
 - Problem: quadratic cost, linear dynamics, no other constraints
 - **Discrete** time, **finite** horizon: perform Riccati recursion
 - **Discrete** time, **infinite** horizon: Discrete Algebraic Riccati equation (DARE)
 - **Continuous** time, **finite** horizon: solve ODE
 - **Continuous** time, **infinite** horizon: Continuous Algebraic Riccati equation (CARE)
 - Variants: time-varying LQR, iterative LQR, tracking LQR,
- HJ reachability (recorded lecture, no exercises)
 - Special setting with zero running cost. Only care about terminal state.

Finite vs infinite time

- Infinite time—when there is no “deadline” to reach the goal
 - Time-varying does not apply
 - Optimal gain is **constant**
- Finite time—want to reach the goal within a fixed time
 - Time-varying is possible
 - Optimal gain is **time-varying**
 - If far from goal and little time left, there is trade-off between control effort to get to goal quickly and not reaching goal

General trajectory optimization problem (Continuous time)

Control signal $\min_{u(\cdot)} \int_0^T J(x(t), u(t), t) dt + J_T(x(T))$ Cost (fuel, distance to goal, rotation,...)

subject to $\dot{x} = f(x(t), u(t), t), t \in [0, T]$ Dynamics

$g(x(t), u(t), t) \leq 0, t \in [0, T]$ Inequality constraints (control and state limits,...)

$h(x(t), u(t), t) = 0, t \in [0, T]$ Equality constraints (initial/goal state,)

Solve HJB equation

Solve (nasty) *PDE* to obtain optimal value function and optimal policy

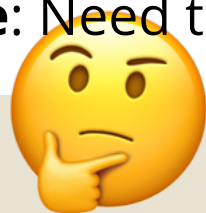
Indirect methods

Apply necessary conditions for optimality (Pontryagin's Maximum Principle). Solve (nasty) *boundary value problem*

Iterative LQR

Approximate problem as LQR problem and repeatedly solve

General theme: Need to solve an *infinite* dimensional optimization problem.



General trajectory optimization problem (Discrete time)

Control sequence $\mathbf{u}$

$$\min_{\mathbf{u}} \sum_{t=0}^{T-1} J(x_t, u_t, t) + J_T(x_T)$$

subject to $x_{t+1} = f(x_t, u_t, t), t = 0, \dots, T - 1$

$$g(\mathbf{x}, \mathbf{u}) \leq 0$$
$$h(\mathbf{x}, \mathbf{u}) = 0$$

Cost (fuel, distance to goal, rotation,...)

Dynamics

Inequality constraints (control and state limits,...)

Equality constraints (initial/goal state,)

Solve Bellman equation

Solve (nasty) *recursion* to obtain optimal value function and optimal policy

Nonlinear program (NLP)

Treat the problem as a mathematical optimization problem.

Iterative LQR

Approximate problem as LQR problem and repeatedly solve

General theme: Need to solve a *finite* dimensional optimization problem.

Nonlinear program

Treat the problem as a mathematical optimization problem.

Let's break this down a bit further...

Continuous time: Optimize then discretize

Discrete time: Discretize then optimize

Transcription

How do we discretize the continuous time problem so an optimizer can solve it?

Shooting
(Single or multiple)

Collocation

Pseudospectral

Single shooting method

- Optimize only for control sequence

Single shooting method

- Optimize only for control sequence
- **States** are **not** decision variables; they are generated by forward integration.
- **Pros:**
 - Problem is simple to implement
 - Good for stable, well-behaved dynamics.
- **Cons:**
 - Very sensitive to dynamics and unstable systems (errors compound).
 - Hard to enforce intermediate state constraints (state must be reached indirectly through controls).
 - Can cause highly nonconvex optimization landscapes.

Multiple shooting method

- Optimize for control sequence and some states

Multiple shooting method

- Optimize for control sequence and some states
- Break horizon into segments: Integrate forward + enforce segments match
Decision variables: controls **and** intermediate “shot” states
- **Pros:**
 - Better numerical conditioning than single shooting.
 - Handles unstable dynamics well.
 - State constraints easy to impose (states are decision variables).
- **Cons:**
 - Larger NLP than single shooting (more variables)
 - Requires enforcing continuity constraints.
 - State constraints can still be tricky

Multiple shooting method

- We could choose a segment with length 1.

$$\min_{\mathbf{u}, \mathbf{x}} \sum_{t=0}^{T-1} J(x_t, u_t, t) + J_T(x_T)$$

$$\text{subject to } x_{t+1} = f(x_t, u_t, t), t = 0, \dots, T - 1$$

$$g(\mathbf{x}, \mathbf{u}) \leq 0$$

$$h(\mathbf{x}, \mathbf{u}) = 0$$

- Easy to write out
- Very common
- Explicitly enforce state constraints as state is a decision variable

Transcription

How do we discretize the continuous time problem so an optimizer can solve it?

Shooting
(Single or multiple)

Single: Forward integrate everything; only controls are optimized

Multiple: Integrate segments + enforce stitching constraints

Collocation

Pseudospectral

Collocation method

- Discretize controls and states, and introduce **algebraic constraints** to enforce the system dynamics at those nodes.
- E.g., Trapezoidal Collocation

$$x_{k+1} = x_k + \frac{h}{2} (f(x_k, u_k) + f(x_{k+1}, u_{k+1}))$$

- Can use more higher-order integration scheme to increased accuracy, or use polynomials
- Big NLP but with nice structure

Collocation method

Pros

- More numerically stable—even for unstable or stiff dynamics
- Easy to impose state and control constraints
- Produces a sparse NLP structure

Cons

- Increased number of decisions variables (states & controls)
- Constraints enforced at discrete nodes
- More complex to formulate

Transcription

How do we discretize the continuous time problem so an optimizer can solve it?

Shooting
(Single or multiple)

Single: Forward integrate everything; only controls are optimized

Multiple: Integrate segments + enforce stitching constraints

Collocation

Discretize states + controls directly; enforce dynamics with algebraic constraints. No forward integration is required.

Pseudospectral

Pseudospectral methods

- Represent control and states as *polynomials*

$$x(t) = \sum_{k=0}^N c_k p_k(t), \quad u(t) = \sum_{k=0}^N d_k p_k(t)$$

Pseudospectral methods

- Represent control and states as *polynomials*

$$x(t) = \sum_{k=0}^N c_k p_k(t), \quad u(t) = \sum_{k=0}^N d_k p_k(t)$$

- Optimize over the *coefficients*, evaluate over nodes
- Easy to differentiate to get $\dot{x}$
- Use “special” polynomial basis and nodes for better numerical stability and properties
 - Chebyshev, Lagrange, Legendre

Pseudospectral method

Pros

- Spectral accuracy (exponential convergence)—if true solution is smooth, then this converges faster
- Better accuracy—number of nodes tradeoff
- Smooth solutions
- Good for long horizon problems

Cons

- Not suitable for problems that require discontinuities
- While less nodes, the matrices can be dense
- Not effective on stiff dynamics
- Complex to implement

Transcription

How do we discretize the continuous time problem so an optimizer can solve it?

Shooting
(Single or multiple)

Single: Forward integrate everything; only controls are optimized

Multiple: Integrate segments + enforce stitching constraints

Collocation

Discretize states + controls directly; enforce dynamics with algebraic constraints. No forward integration is required.

Pseudospectral

Represent states and controls as a polynomial and optimize for polynomial coefficients instead.

Solving the NLP

Great! Now I have a NLP, how do I solve it?

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{x}} \quad & \sum_{t=0}^{T-1} J(x_t, u_t, t) + J_T(x_T) \\ \text{subject to} \quad & x_{t+1} = f(x_t, u_t, t), t = 0, \dots, T-1 \\ & g(\mathbf{x}, \mathbf{u}) \leq 0 \\ & h(\mathbf{x}, \mathbf{u}) = 0 \end{aligned}$$

Many solvers exist!

<https://uw-ctrl.github.io/lmc-book/lectures/optimization.html>

NLP solvers

- Use a nonlinear program solver
- Matlab fmincon

Description

Nonlinear programming solver.

Finds the minimum of a problem specified by

$$\min_x f(x) \text{ such that } \begin{cases} c(x) \leq 0 \\ ceq(x) = 0 \\ A \cdot x \leq b \\ Aeq \cdot x = beq \\ lb \leq x \leq ub, \end{cases}$$

NLP solvers

- Use a nonlinear program solver
- Matlab `fmincon`
- `scipy.optimize.minimize`

```
scipy.optimize.  
minimize
```

```
minimize(fun, x0, args=(), method=None, jac=None, hess=None, hessp=None,  
bounds=None, constraints=(), tol=None, callback=None, options=None)
```

Minimization of scalar function of one or more variables.

[\[source\]](#)

NLP solvers

- Use a nonlinear program solver
- Matlab `fmincon`
- `scipy.optimize.minimize`
- Many others, commercial solvers, free academic licenses.
- Lots of open-source ones too

But what is happening under the hood in these NLP solvers?

- Really smart with linear algebra and other tricks
 - E.g., exploiting sparsity, well-conditioning the problem
- Some sort of **log-barrier method**, turning constrained to unconstrained
- Apply some form of approximate method
- Smarter gradient descent
 - Use second order information, smarter line search

Sequential quadratic programming

- Approximate the problem as a quadratic program and repeatedly solve
 1. Start with initial guess $(\tilde{x}_0, \tilde{x}_1, \dots, \tilde{x}_{T+1}, \tilde{u}_0, \tilde{u}_1, \dots, \tilde{u}_T)$
 2. Linearize dynamics about guess $x_{t+1} \approx A_t x_t + B_t u_t + C_t$
 3. Linearize constraints $g(x, u) \leq 0$: $G_t x_t + H_t u_t + h_t \leq 0$
 4. Quadratize cost $c(x_t, u_t) \approx \frac{1}{2} x_t^T Q_t x_t + \frac{1}{2} u_t^T R_t u_t + p_t^T x_t + r_t^T u_t$
 5. Add trust region penalty to cost: $x_t \approx \tilde{x}_t, u_t \approx \tilde{u}_t$
 6. Solve QP! Solution = new guess
 7. Go to step 1.

1. Start with initial guess
 $(\tilde{x}_0, \tilde{x}_1, \dots, \tilde{x}_{T+1}, \tilde{u}_0, \tilde{u}_1, \dots, \tilde{u}_T)$
2. Linearize dynamics about guess
3. Linearize constraints
4. Quadratize cost
5. Add trust region penalty to cost:
6. Solve QP! Solution = new guess
7. Go to step 1.

What if the problem is *really* hard?



Or when there's lots of dynamic obstacles



What are the challenge that arise?

- Cost, dynamics, constraints are *ugly* to work with
 - Difficult to differentiate
 - Maybe blackbox
 - Maybe learned (neural network)
- Stochastic elements
 - Uncertainty in the environment.
 - Require drawing many samples
- Generally, the standard NLP / gradient descent framework is not applicable.

Sampling-based: Model Predictive Path Integral

Information Theoretic MPC for Model-Based Reinforcement Learning

Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews,
James M. Rehg, Byron Boots, and Evangelos A. Theodorou

Abstract—We introduce an information theoretic model predictive control (MPC) algorithm capable of handling complex cost criteria and general nonlinear dynamics. The generality of the approach makes it possible to use multi-layer neural networks as dynamics models, which we incorporate into our MPC algorithm in order to solve model-based reinforcement learning tasks. We test the algorithm in simulation on a cart-pole swing up and quadrotor navigation task, as well as on actual hardware in an aggressive driving task. Empirical results demonstrate that the algorithm is capable of achieving a high level of performance and does so only utilizing data collected from the system.



Fig. 1. Aggressive driving with MPPI and neural network dynamics.

Sampling-based: Model Predictive Path Integral

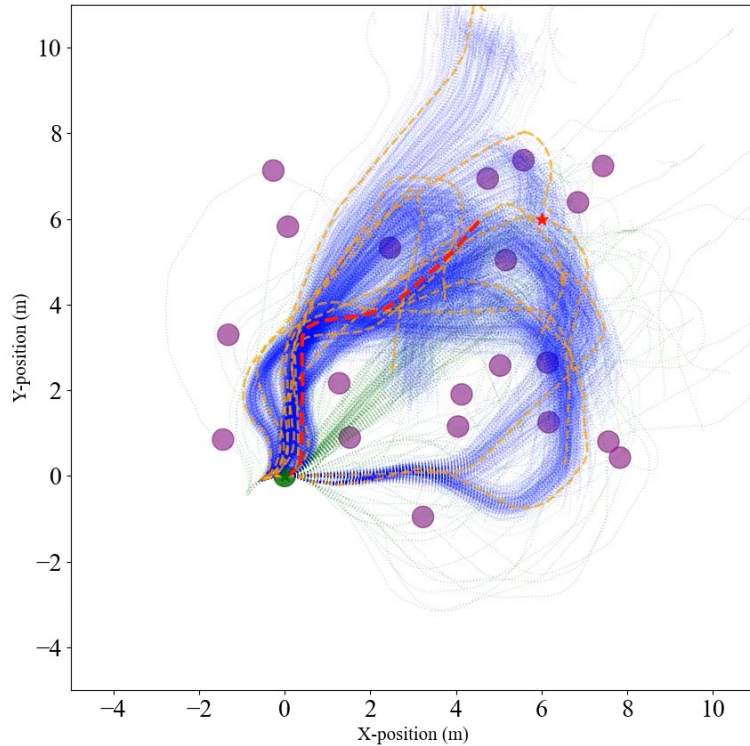
1. Start with nominal trajectory $\tilde{\mathbf{u}}$
2. Add noise to it to generate many trajectories $\boldsymbol{\delta}_k$
3. Evaluate cost of each trajectory c_k
4. Compute weight for each trajectory $w_k = \frac{e^{c_k}}{\sum_j e^{c_j}}$
5. Compute weighted sum over controls to compute control $\mathbf{u}^* = \tilde{\mathbf{u}} + \sum_k w_k \boldsymbol{\delta}_k$

Idea: Take a noisy estimate of the gradient and move along that direction

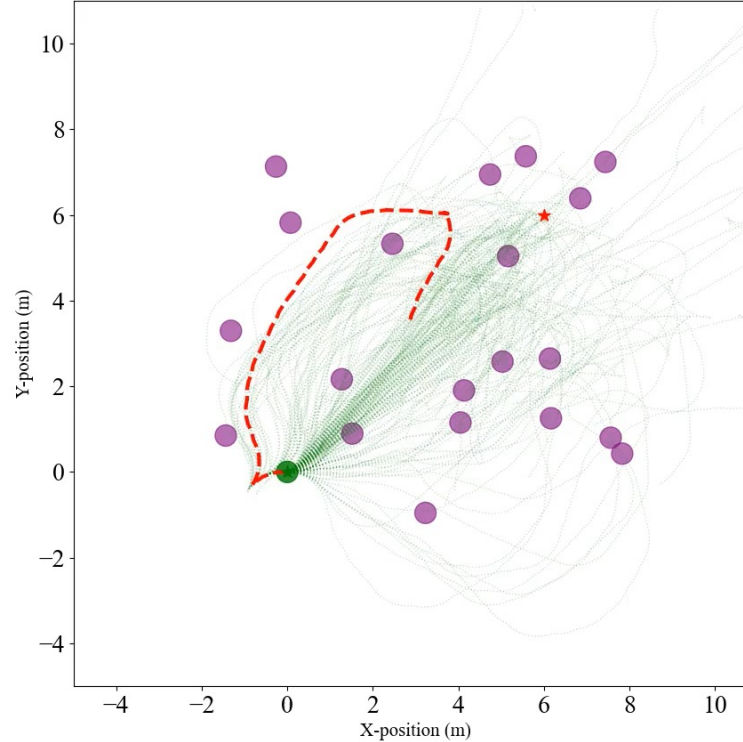
1. Start with nominal trajectory $\tilde{\mathbf{u}}$
2. Add noise to it to generate many trajectories δ_k
3. Evaluate cost of each trajectory c_k
4. Compute weight for each trajectory
$$w_k = \frac{e^{-c_k}}{\sum_j e^{-c_j}}$$
5. Compute weighted sum over controls to compute control $\mathbf{u}^* = \tilde{\mathbf{u}} + \sum_k w_k \delta_k$

Combining generative modeling with MPPI

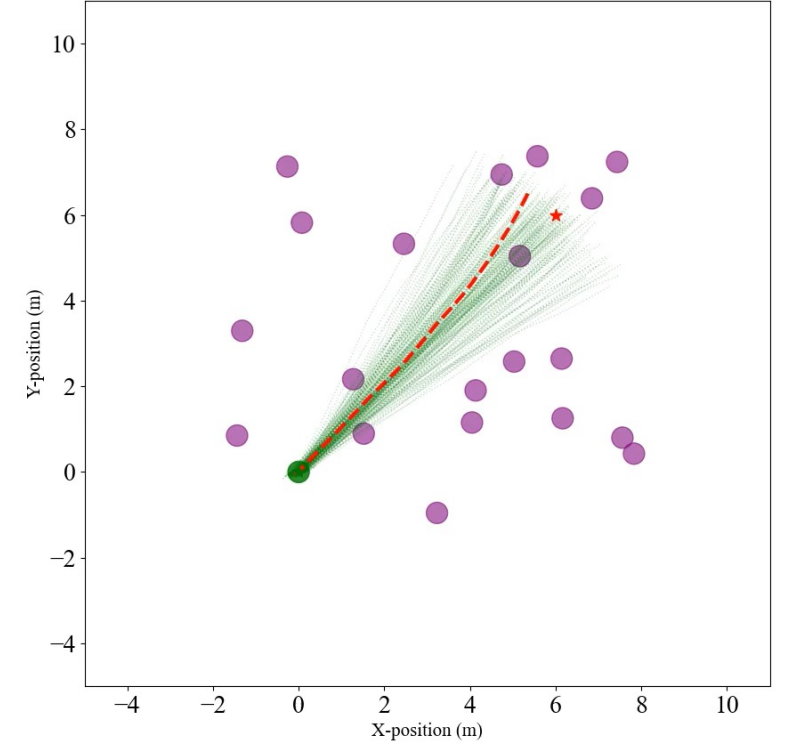
Generative AI + optimization



Generative AI only



Optimization only



Explores multiple options and picks the best one

Model Predictive Control

Middle ground between open-loop and closed-loop?

Open-loop control

Planning a trajectory/control sequence at time zero.

Plan becomes invalid very quickly

Over the entire, potentially very long, horizon

“Easier” to compute

Model Predictive Control (MPC)

Continually recompute an open-loop trajectory at each time step.

Solve an OCP problem over a shorter horizon

Update problem parameters as new information is received

“Open-loop control with feedback”

Closed-loop control

Very difficult to solve for stochastic, high dimensional, nonlinear problems.

Computed offline. Minimal computation online.

Invalid if there's model mismatch

Give optimal policy

Researchers program an autonomous vehicle to drift around obstacles using Nonlinear Model Predictive Control (NMPC)



Bram Geenen
19 Jul, 2022

 FOLLOW



image: Toyota Research Institute

Obstacle appearing, tire friction changing, road conditions changing

<https://www.wevolver.com/article/researchers-program-an-autonomous-vehicle-to-drift-around-obstacles-using-nonlinear-model-predictive-control-nmpc>

PAMPC: Perception-Aware Model Predictive Control for Quadrotors



Unknown perception quality beforehand. Only known during deployment.

Falanga et al, IROS 2018
<https://arxiv.org/pdf/1804.04811.pdf>

Quadruped trajectory optimization

Highly Dynamic Quadruped Locomotion via Whole-Body Impulse Control and Model Predictive Control

Donghyun Kim¹, Jared Di Carlo², Benjamin Katz¹, Gerardo Blede¹, and Sangbae Kim¹

Abstract—Dynamic legged locomotion is a challenging topic because of the lack of established control schemes which can handle aerial phases, short stance times, and high-speed leg swings. In this paper, we propose a controller combining whole-body control (WBC) and model predictive control (MPC). In our framework, MPC finds an optimal reaction force profile over a longer time horizon with a simple model, and WBC computes joint torque, position, and velocity commands based on the reaction forces computed from MPC. Unlike existing WBCs, which attempt to track commanded body trajectories, our controller is focused more on the reaction force command, which allows it to accomplish high speed dynamic locomotion with aerial phases. The newly devised WBC is integrated with MPC and tested on the Mini-Cheetah quadruped robot. To demonstrate the robustness and versatility, the controller is tested on six different gaits in a number of different environments, including outdoors and on a treadmill, reaching a top speed of 3.7 m/s.

I. INTRODUCTION

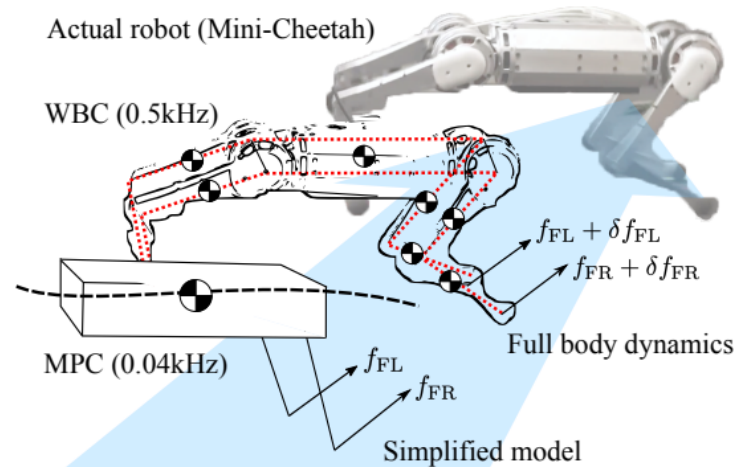


Fig. 1. **Control Architecture.** The proposed control architecture consists of two parts: Model predictive control and whole-body control. The reaction forces computed by MPC are modified by WBC to incorporate body stabilization and swing leg control. The final commands found in WBC are sent to the robot to perform dynamic locomotion.

Contact dynamics are hard!
Need to adapt to the terrain
and rigid body dynamics.

MPC Core Idea

Repeatedly solve an optimization problem over a finite horizon, execute first control, replan at next time step with updated state (and environment) information

General set-up

- Suppose we want to plan over a long time horizon (T very large, or potentially infinite)
 - E.g., chemical plant that needs to operate continuously
 - Not tractable to compute over such a long time horizon
- You can't compute the optimal policy (i.e., compute value function) offline
- Your model isn't perfect, and you don't trust your open-loop solution

Receding horizon planning / MPC

1. Plan a trajectory over a **finite horizon** (i.e., a sequence of control inputs)
 2. Execute the **first control input** in the sequence
 3. Go to step 1.
- Solve long horizon problem with a sequence of shorter horizon problem.
 - Just make T smaller...right?
 - **Potential issue?**

Potential challenges

- Limited prediction horizon: MPC planning horizon is often not long enough to reach the end goal (if it exists).
- May lead to a situation where after a few steps, the MPC problem becomes infeasible
- Persistent feasibility?
 - Ensure that future problems will be feasible
- Closed-loop stability?
 - Ensure that in the future, the system will eventually converge to the goal state

Need some “foresight” to account for the future timesteps

- Let N be the shorter planning horizon
- Let t be the current time.
- We are planning from t to $t + N$

$$\min_{u=(u_{t|t}, \dots, u_{t+N|t})} \sum_{k=0}^{N-1} g(x_{t+k|t}, u_{t+k|t}) + p(x_{t+N|t})$$

Cost-to-go

$$\text{s. t. } x_{t+k+1|t} = f(x_{t+k|t}, u_{t+k|t})$$

$$x_{t+k|t} \in X, u_{t+k|t} \in U \quad \forall k$$

$$x(t) = x_{t|t}$$

Terminal set
constraint

$$x_{t+N|t} \in X_f$$

Something to ensure that
the system will end up in
a “reasonable” state

Persistent feasibility theorem

- If X_f is a *control invariant set* for the system $x_{k+1} = f(x_k, u_k)$ then the MPC law is persistently feasible. i.e., the next planning step will have a feasible solution

$$\begin{aligned} \min_{u=(u_{t|t}, \dots, u_{t+N|t})} & \sum_{k=0}^{N-1} g(x_{t+k|t}, u_{t+k|t}) + p(x_{t+N|t}) \\ \text{s. t. } & x_{t+k+1|t} = f(x_{t+k|t}, u_{t+k|t}) \\ & x_{t+k|t} \in X, u_{t+k|t} \in U \quad \forall k \\ & x(t) = x_{t|t} \\ & x_{t+N|t} \in X_f \text{ Control invariant set} \end{aligned}$$

Proof (sketch)

- Suppose at time t , you have a feasible solution to the current planning problem. $\mathbf{u}_0^* = (u_0, u_1, \dots, u_{N-1})$ and $x_N \in X_f$
- At the next planning step (shift time step forward by one), note that $\mathbf{u}_1 = (u_1, u_2, \dots, u_{N-1}, \tilde{u})$ is a feasible solution to the next planning problem where $\tilde{x}_{N+1} = f(x_N, \tilde{u}) \in X_f$ ($\tilde{u}$ is a feasible control corresponding to the control invariant set definition)
- We have just found a feasible solution to the next planning problem. So we will do at least as good, if not better at the next time step.

Practical considerations

- The terminal set X_f is introduced artificially for the sole purpose of leading to a sufficient condition for persistent feasibility
- Want it to be large so it does not compromise on performance
- How do we pick X_f ?
- These sets can be computed by using the MPT toolbox
<https://www.mpt3.org/>

MPC stability theorem

Assume:

1. X, U contains the origin in their interior
2. $x = 0, u = 0$ is an equilibrium $f(0,0) = 0$
3. $\beta^- \|x\| \leq g(x, u) \leq \beta^+ \|x\| \quad \forall x \in X, u \in U$
4. $X_f \subseteq X$ is control invariant
5. The terminal cost p is a local control Lyapunov function satisfying
 - $p(x) \leq \beta \|x\|$
 - $\exists \mu \quad p(x) - p(f(x, \mu(x))) \geq g(x, \mu(x)). \quad \forall x \in X_f$

Then the origin of the closed-loop system under MPC control is exponentially stable with region of attraction X_0 (N -step BRS of X_f)

Proof (sketch)

- Show that the cost is Lyapunov stable for the closed-loop system
 - i.e., the cost is shrinking over time
- Suppose you have an optimal trajectory $\mathbf{u}_0^* = (u_0, u_1, \dots, u_{N-1})$
- At the next time step $\mathbf{u}_1 = (u_1, u_2, \dots, u_{N-1}, \tilde{u})$ ($\tilde{u}$ keep $x_{N+1} \in X_f$)
- But $\mathbf{u}_1$ is not necessarily optimal

Rough proof continued

$$\begin{aligned} J_1^*(x_1) &\leq \sum_{k=1}^N g(x_k, u_k) + p(x_{N+1}) \\ &= \sum_{k=1}^{N-1} g(x_k, u_k) + g(x_N, \tilde{u}) + p(x_{N+1}) \end{aligned}$$

$$= \sum_{k=1}^{N-1} g(x_k, u_k) + g(x_N, \tilde{u}) + p(x_{N+1}) + p(x_N) - p(x_N) + g(x_0, u_0) - g(x_0, u_0)$$

$$J_1^*(x_1) \leq J_0^*(x_0) + g(x_N, \tilde{u}) + p(x_{N+1}) - p(x_N) - g(x_0, u_0)$$

$$J_0^*(x_0) - J_1^*(x_1) \geq g(x_0, u_0) \geq \beta^- \|x_0\|$$

MPC stability theorem takeaway

- If we choose the cost-to-go to be a control Lyapunov function, then we can ensure stability (with some assumptions about the problem structure)

$$\begin{aligned} \min_{\mathbf{u}=(u_{t|t}, \dots, u_{t+N|t})} & \sum_{k=0}^{N-1} g(x_{t+k|t}, u_{t+k|t}) + p(x_{t+N|t}) \\ \text{s. t. } & x_{t+k+1|t} = f(x_{t+k|t}, u_{t+k|t}) \\ & x_{t+k|t} \in X, u_{t+k|t} \in U \quad \forall k \\ & x(t) = x_{t|t} \\ & x_{t+N|t} \in X_f \text{ Control invariant set} \end{aligned}$$

Control
Lyapunov
function

Some comments

- At present there is no other technique than MPC to design controllers for general large linear multivariable systems with input and output constraints with a stability guarantee
- Useful toolbox (MATLAB): <https://www.mpt3.org/>
- In practice, sometimes choosing a good terminal cost is enough (i.e., don't need to enforce a terminal control invariant condition), though you may be sacrificing guarantees

So far...

- Assumed perfect dynamics model
- Assumed no stochasticity
- What about moving obstacles? (Prediction models, human-robot interactions)
- Theorems hold in general but challenging for non-LQR-like structure (i.e., not linear & not quadratic)
- But provide conceptual understanding to what heuristics we would like to design for nonlinear systems
- There is a whole research area on *robust* MPC and *tube* MPC (beyond the scope of this course)

Code demo

Planning a trajectory to avoid moving asteroids

